gomesf@chalmers.se
Computer Science and Engineering Department
Chalmers and the University of Gothenburg

DAT321 DIT843
Internal Quality – Code Metrics

Francisco Gomes de Oliveira Neto

(he/him)

Barbara Liskov

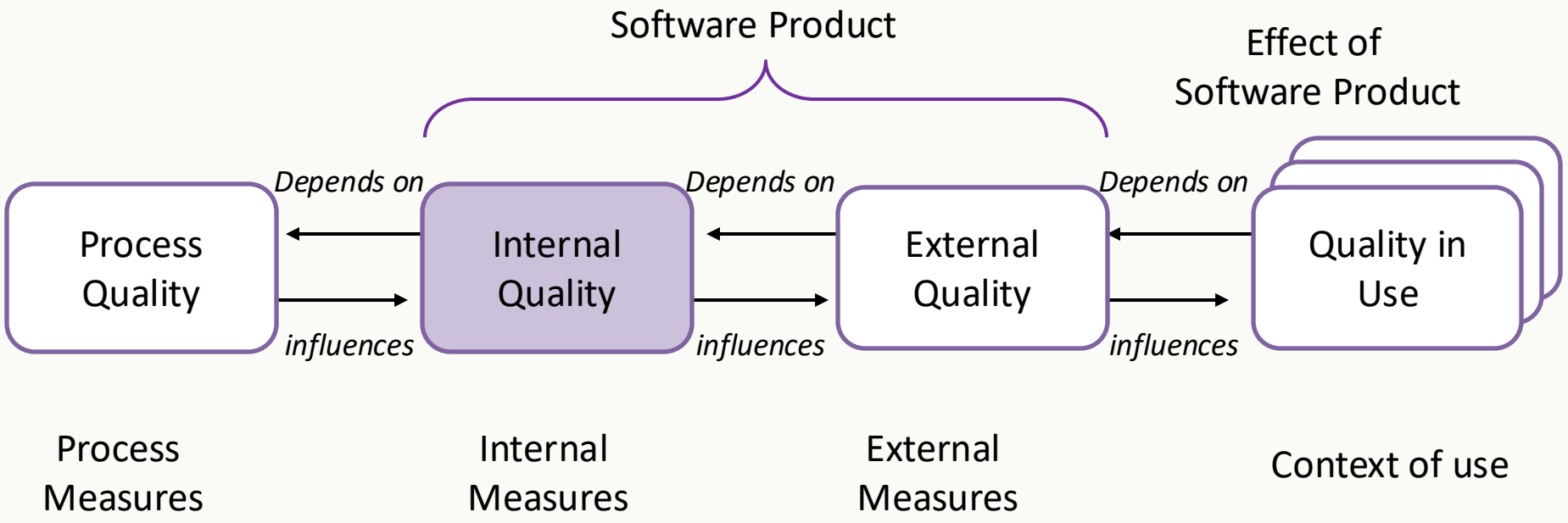
- One of the pioneers in distributed computing
- Author of the Liskov Substitution Principle (A pillar of OOP)
- Turing Award Winner 2008

What will we learn today?

- What is **Internal Quality**? How to **measure** Internal Quality?
- What is **Refactoring**? What are **code smells**?
- What are the different metrics for **size, complexity, coupling**?
- Can **Large Language Models (AI) measure** internal quality?

© Francisco Oliveira Neto - 2018

Software Quality – Internal Quality



Source: International Standard Organization, ISO/IEC 25010 standard

© Francisco Oliveira Neto - 2018

What is Refactoring?

- Refactoring is to improve the structure of existing code **without changing its external behavior**.

  Refactoring is NOT bug fixing!! Can you explain why?

- **We refactor to** simplify complex code, improve readability, reduce duplication, improve efficiency.
- **We refactor** before implementing new features, or during code reviews.

What to do with measures?

- We assess the internal quality to understand how close the actual design of a particular artifact is to the **optimal design**
- Solution for low internal quality: **Refactoring**

- **Why** refactoring is exclusive to internal quality?
  - Because **refactoring can never change behavior**. Hence, only affects the static properties of the software product.

Code Smells

Indications that something is wrong with the code, but not necessarily bugs.

- **Magic numbers:** Hardcoded values in code
  - Fix: Use constants to improve readability
- **Duplicated Code:** Same logic repeated in different places.
  - Fix: Use functions to reuse code.
- **Long Functions:** Functions that do too many things when they shouldn’t.
  - Fix: Split into smaller, focused functions.
- **Large Parameter Lists:** Too many function parameters.
  - Fix: Use tuples, lists, or objects to group related data.
- **Inconsistent Naming:** Variable/function names don't describe their purpose.
  - Fix: Choose clear, consistent names and follow conventions.
- **The Blob:** A class or file that concentrates a lot of functionalities.
  - Fix: Divide the code into packages and different classes.

Some examples of Code Measures

- Lines of code
- Code churn
- Block depth
- Fan-in and Fan-out
- Cyclomatic Complexity
- Cognitive Complexity

© Francisco Oliveira Neto - 2018

Part 1:
Code Size and Dependency

© Francisco Oliveira Neto - 2018

```java
616     @Override
617     public void write(final byte[] buf, int offs, final int len)
618         throws IOException {
619         if (offs < 0) {
620             throw new IndexOutOfBoundsException("offs(" + offs + ") < 0.");
621         }
622         if (len < 0) {
623             throw new IndexOutOfBoundsException("len(" + len + ") < 0.");
624         }
625         if (offs + len > buf.length) {
626             throw new IndexOutOfBoundsException("offs(" + offs + ") + len("
627                                                 + len + ") > buf.length("
628                                                 + buf.length + ").");
629         }
630         if (closed) {
631             throw new IOException("Stream closed");
632         }
633
634         for (final int hi = offs + len; offs < hi;) {
635             write0(buf[offs++]);
636         }
637     }
```

How many Lines of Code? (LOC)

a) 21 b) 11 c) 12 d) 22

Software Size: Lines of Code (LOC)

- Lines of Code (LOC): Number of **statements** in a piece of code.
  - Skip comments and blank lines. Often measures in **kLOC**.
- Physical vs. Logical Lines of Code.
  - Physical: Actual lines in the file
  - Logical: Number of code statements (independently of layout)
  - **Logical** lines of code is more used.
- Commented Lines: Number of lines with comments
  - The **ratio** between LOCs and Lines with Comments shows documentation effort.
  - `Comments (%) = CommentedLines / (LOCs + CommentedLines)`

© Francisco Oliveira Neto - 2018

Software Size: Lines of Code (LOC)

- Example 1:

```
/* How many lines of code is this?*/
for (i = 0; i < 100; i++) printf("hello");
```

```
- 1 physical LOC;
- 2 logical LOC; <--
```

- Example 2:

```
public HashCodeBuilder append(final boolean[] array) {
    if (array == null) {
        iTotal = iTotal * iConstant;
    } else {
        for (final boolean element : array) {
            append(element);
        }
    }
    return this;
}
```

```
- 10 physical LOC;
- 6 logical LOC; <--
    - Method, if-else, iTotal,
      for, append, return
```


© Francisco Oliveira Neto - 2018

Software Size: Lines of Code (LOC)

- By itself, **LOCs are not informative**. Can also vary depending on the programming language.
  - Example: Java is more verbose than Python, so Java programs tend to be larger than Python programs.
  - Does this mean the Java programs are really “bigger” than Python programs?
- LOCs should be associated with **other measures** (e.g., LOC per method or commit)
- **Code churn:** `LOCs Added - LOCs Removed`
  - Measured over a **period of time**, (e.g., a week, a day, over a sprint).
  - High (positive) code churn over a long time indicates not enough **refactoring**.

© Francisco Oliveira Neto - 2018

Summary: Lines of Codes (LOC)

- Two ways to calculate LOCS: Physical, or Logical:
- By itself, not very informative.
- Better to measure in kLOC.

```
- 1 physical LOC;
- 2 logical LOC; <--
```

```
/* How many lines of code is this?*/
for (i = 0; i < 100; i++) printf("hello");

 public HashCodeBuilder append(final boolean[] array) {
     if (array == null) {
         iTotal = iTotal * iConstant;
     } else {
         for (final boolean element : array) {
             append(element);
         }
     }
     return this;
 }
```

```
- 10 physical LOC;
- 6 logical LOC; <--
    - Method, if-else, iTotal,
      for, append, return
```


© Francisco Oliveira Neto - 2018

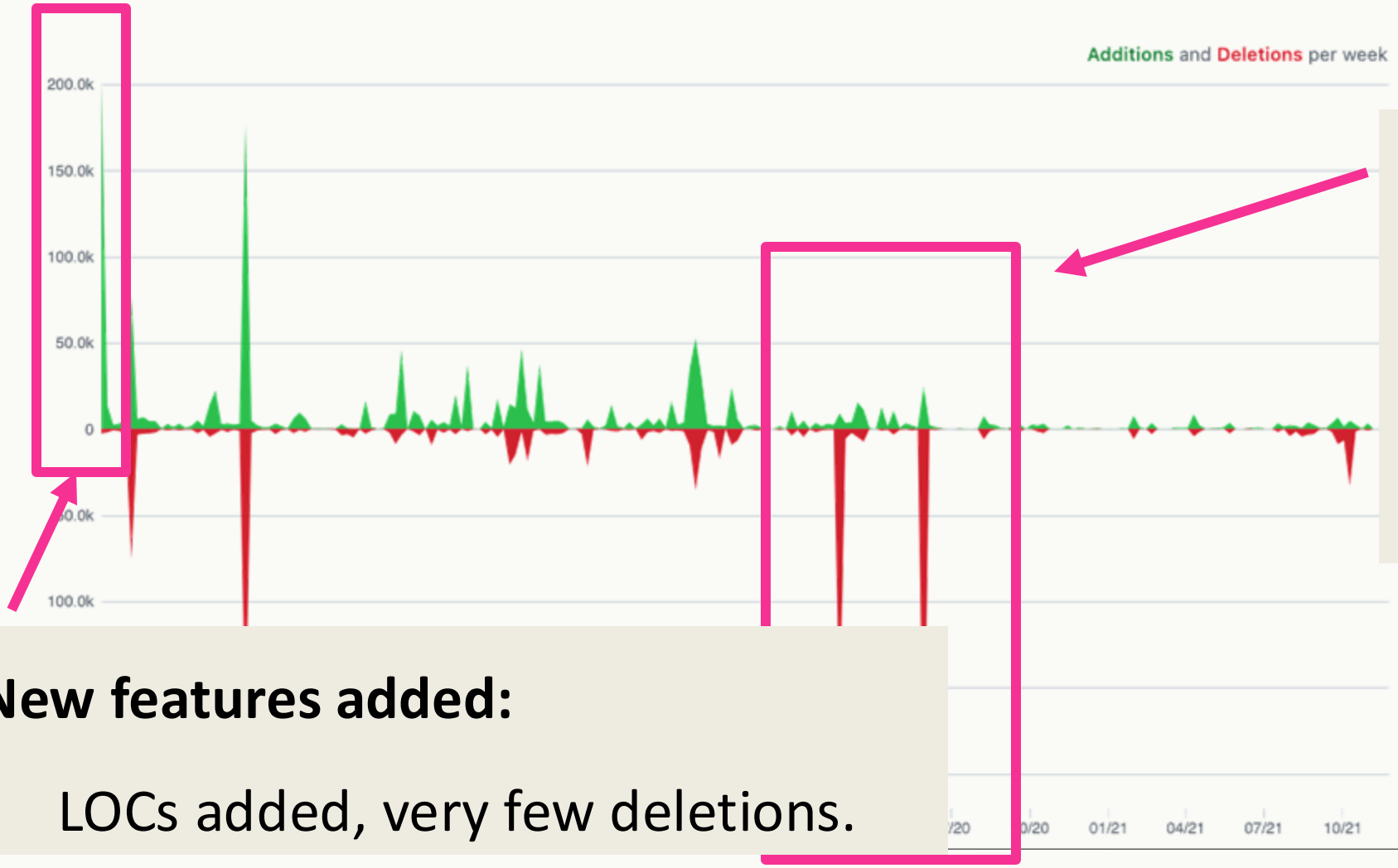
Src: https://github.com/Netflix/titus-control-plane/graphs/code-frequency

Software Size: Code Churn

© Francisco Oliveira Neto - 2018

Src: https://github.com/Netflix/titus-control-plane/graphs/code-frequency

Software Size: Code Churn



Refactoring sessions

- Few LOCs added with a lot of deletions.

New features added:

- LOCs added, very few deletions.

© Francisco Oliveira Neto - 2018

Software Size: Block Depth

- Block **Depth**: Number of **nested** code statements.
- Deeper blocks are much **harder to understand**, specially if control flow (if-else, while or for loops) constructs are nested.
- Also **affects complexity** measures (Cognitive and Cyclomatic)
- Depth should be less than 6. **Not more!**

© Francisco Oliveira Neto - 2018

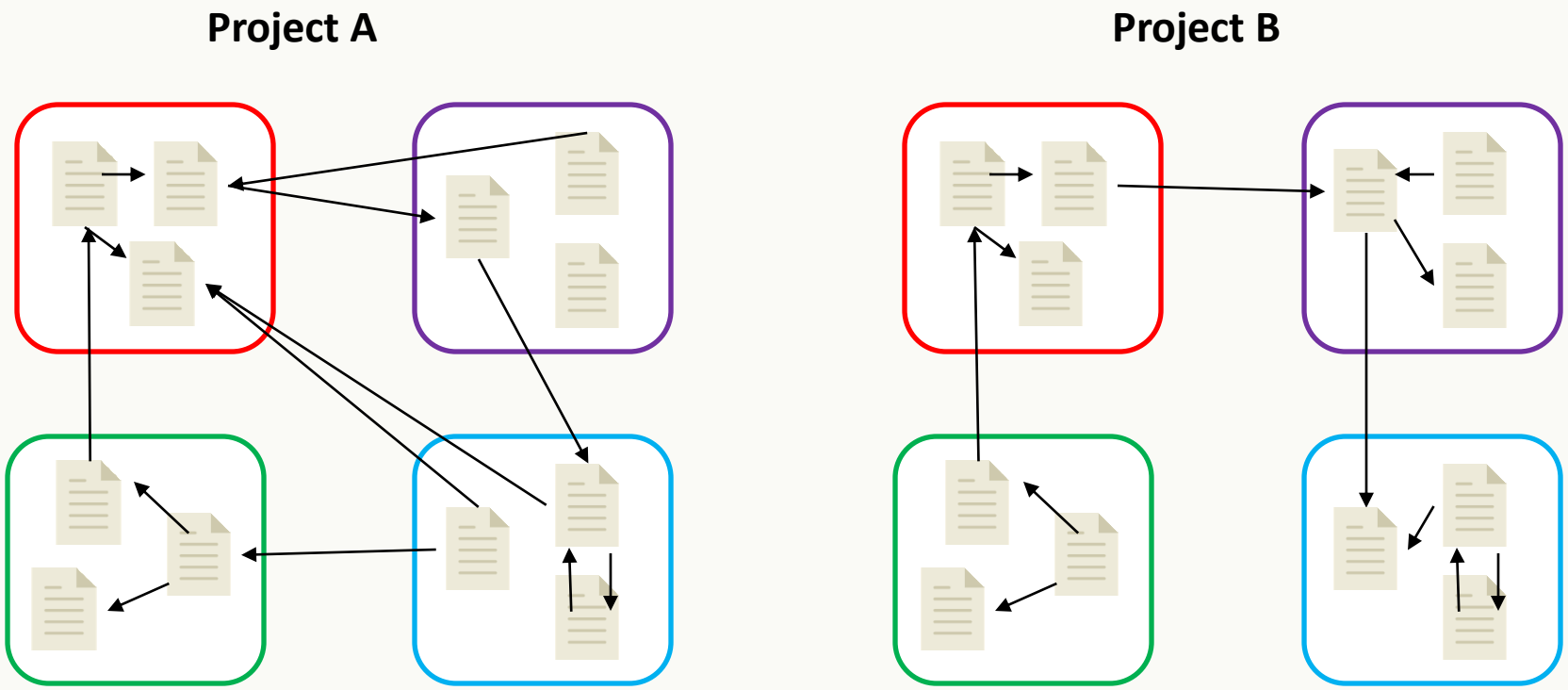
Software Size: Block Depth

```
141 @   public static Fraction getFraction(int numerator, int denominator) {
142 1       if (denominator == 0) {
143 2           throw new ArithmeticException("The denominator must not be zero");
144         }
145         if (denominator < 0) {
146 2           if (numerator == Integer.MIN_VALUE || denominator == Integer.MIN_VALUE) {
147 3               throw new ArithmeticException("overflow: can't negate");
148             }
149             numerator = -numerator;
150             denominator = -denominator;
151         }
152         return new Fraction(numerator, denominator);
153     }
```

Highest block depth is 3.

© Francisco Oliveira Neto - 2018

Coupling measures



Which has better **modularity**?

What would happen if we **change** one class?

Project B is better, as there are fewer inter- dependencies, hence minimising the external impact of changing one class.

Coupling: Fan-in and Fan-out

- Coupling and cohesion. Ideally: High cohesion + Low coupling.
  - Be careful: **Intra**-dependencies vs. **Inter**-dependencies.
  - More inter-dependencies is worse, as changes can propagate further.
- **Fan-in** of M: Number of modules calling M.
- **Fan-out** of M: Number of modules called by M.
  - Modules here can be packages, functions, classes, etc.

Coupling: Fan-in, Fan-out and HK

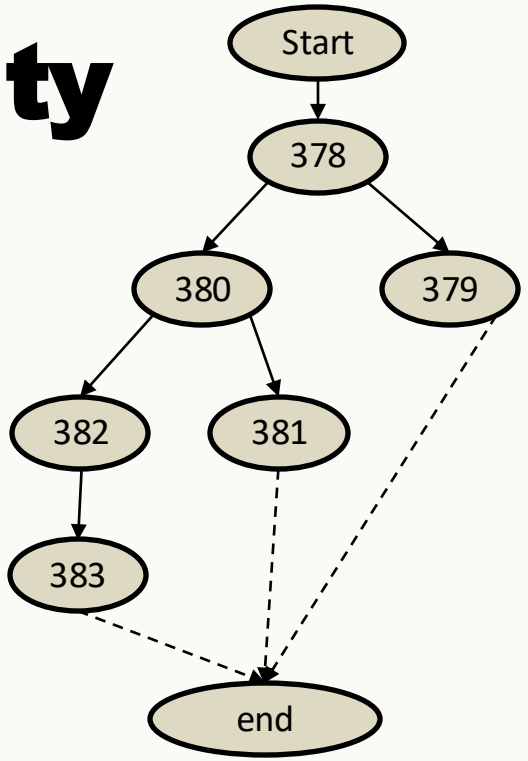
- **High fan-in** of method / class X, means that changing **X impacts** many parts of the code.
- **High fan-out** of method / class X, means that **X is impacted** by changing many parts of the code.

Is it better to change a method with high fan-in or high fan-out?

Part 2:
Code Understandability

© Francisco Oliveira Neto - 2018

Complexity: Cyclomatic Complexity



- Known as McCabe's **cyclomatic complexity.** Measures the number of **linearly independent paths** in a function. Conveys complexity of the control flow.
- Uses a control flow graph: **M = E – N + 2P.**
- E – the number of edges
- N – the number of nodes
- M – McCabe complexity number
- P – Programs (e.g., num of functions)

M = 9 - 8 + 2 = **3**

```
377 public static void isLeapYear(int year){
378     if (year < 0) {
379         System.out.println("Year cannot be negative.");
380     } else if (((year % 4 == 0) && (year % 100!= 0)) || (year%400 == 0)) {
381         System.out.println("Specified year is a leap year");
382     } else {
383         System.out.println("Specified year is not a leap year");
384     }
385 }
```


© Francisco Oliveira Neto - 2018

Calculate the Cyclomatic Complexity of the pow function:

```java
535     public Fraction pow(final int power) {
536         if (power == 1) {
537             return this;
538         }
539         if (power == 0) {
540             return ONE;
541         }
542         if (power < 0) {
543             if (power == Integer.MIN_VALUE) { // MIN_VALUE can't be negated.
544                 return this.invert().pow(2).pow(-(power / 2));
545             }
546             return this.invert().pow(-power);
547         }
548         final Fraction f = this.multiplyBy(this);
549         if (power % 2 == 0) { // if even...
550             return f.pow(power / 2);
551         }
552         return f.pow(power / 2).multiplyBy(this);
553     }
```

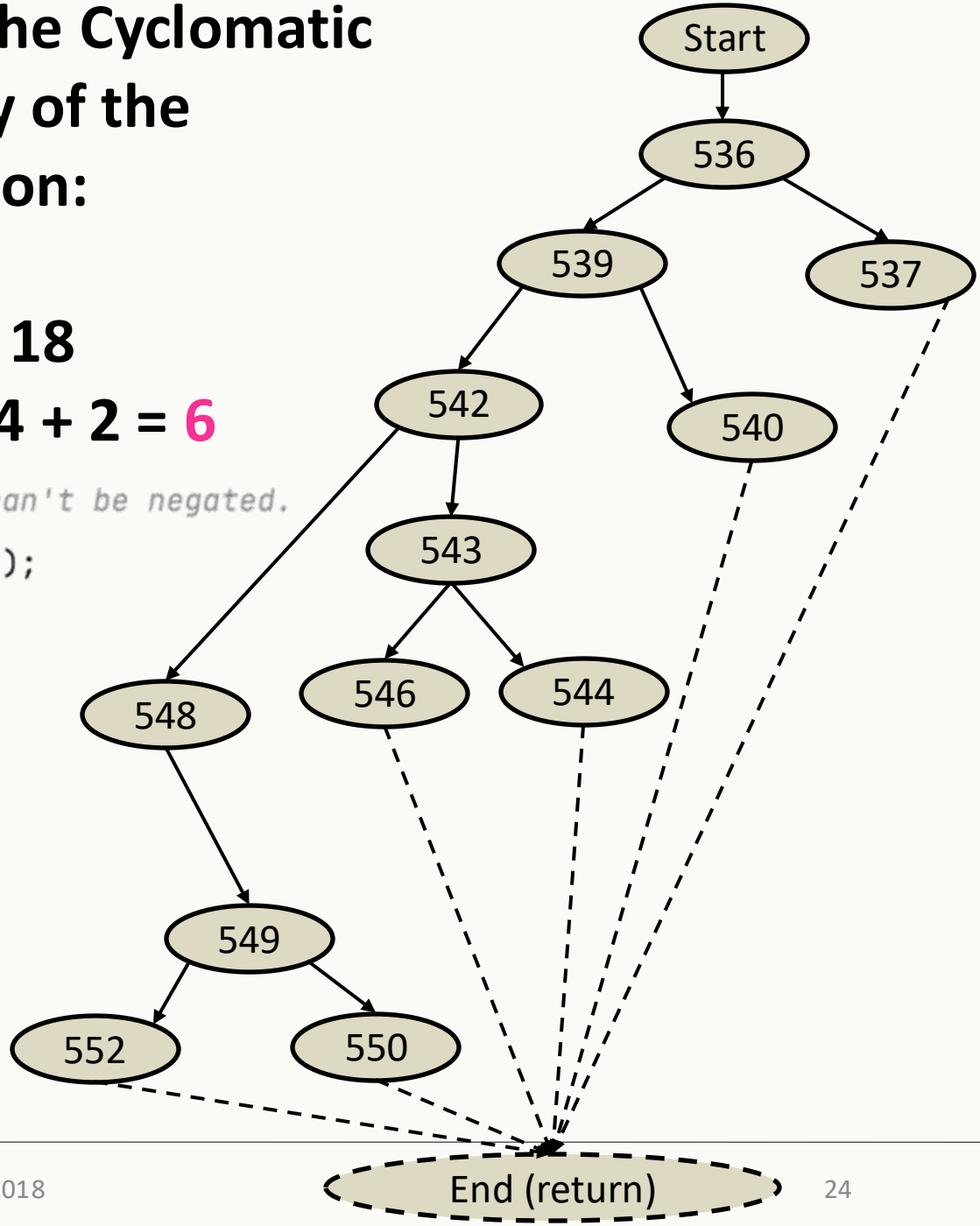

© Francisco Oliveira Neto - 2018

```
535 public Fraction pow(final int power) {
536     if (power == 1) {
537         return this;
538     }
539     if (power == 0) {
540         return ONE;
541     }
542     if (power < 0) {
543         if (power == Integer.MIN_VALUE) { // MIN_VALUE can't be negated.
544             return this.invert().pow(2).pow(-(power / 2));
545         }
546         return this.invert().pow(-power);
547     }
548     final Fraction f = this.multiplyBy(this);
549     if (power % 2 == 0) { // if even...
550         return f.pow(power / 2);
551     }
552     return f.pow(power / 2).multiplyBy(this);
553 }
```

Calculate the Cyclomatic Complexity of the pow function:

N = 14; E = 18

M = 18 – 14 + 2 = 6



© Francisco Oliveira Neto - 2018

Complexity: Cognitive Complexity

- The code below have the same cyclomatic complexity. Are they **“equaly” complex**?

```java
int sumOfPrimes(int max) {                    // +1
    int total = 0;
    OUT: for (int i = 1; i <= max; ++i) { // +1
        for (int j = 2; j < i; ++j) {     // +1
            if (i % j == 0) {             // +1
                continue OUT;
            }
        }
        total += i;
    }
    return total;
} // Cyclomatic Complexity 4
```

```java
String getWords(int number) { // +1
    switch (number) {
        case 1: // +1
            return "one";
        case 2: // +1
            return "a couple";
        case 3: // +1
            return "a few";
        default:
            return "lots";
    }
} // Cyclomatic Complexity 4
```

Complexity: Cognitive Complexity

- Proposed by the SonarSource team. **Simplified version of cyclomatic** complexity. Considers **nested** blocks and **complex** operations.
- Conveys the **comprehensibility** of a program. How hard it is to understand the output of a code. Rules:
  - Ignore structures that allow multiple statements easier to read (e.g., switch).
  - **+1** complexity for each break in the **linear flow** of the code
  - **+1** complexity when flow-breaking **structures are nested.**

© Francisco Oliveira Neto - 2018

Complexity: Cognitive Complexity

```
void myMethod () {
  try {
    if (condition1) {                    // +1
      for (int i = 0; i < 10; i++) {     // +2 (nesting=1)
        while (condition2) { … }         // +3 (nesting=2)
      }
    }
  } catch (ExcepType1 | ExcepType2 e) { // +1
    if (condition2) { … }                // +2 (nesting=1)
  }
}                                        // Cognitive Complexity 9
```

From: COGNITIVE COMPLEXITY. G. Ann Campbell. 2016. page 9

© Francisco Oliveira Neto - 2018

Complexity: Cognitive Complexity

- The codes from the initial example have **different cognitive complexity**.

```java
int sumOfPrimes(int max) {
    int total = 0;
    OUT: for (int i = 1; i <= max; ++i) { // +1
        for (int j = 2; j < i; ++j) {     // +2
            if (i % j == 0) {             // +3
                continue OUT;             // +1
            }
        }
        total += i;
    }
    return total;
} // Cognitive Complexity = 7
```

```java
String getWords(int number) {
    switch (number) {
        case 1:                 // +1
            return "one";
        case 2:
            return "a couple";
        case 3:
            return "a few";
        default:
            return "lots";
    }
} // Cognitive Complexity = 1
```

What about Generative AI:

- **Can ChatGPT understand code complexity?** Prompt it:

  Option 1: Give code, ask its cyclomatic complexity

  Option 2: Explain how to calculate cyclomatic complexity, give example with result, then give the code.

- Reflections: **Which results did you get? What do you think of its answers?**

© Francisco Oliveira Neto - 2018

What about Generative AI:

Links for Chat with ChatGPT during Lecture:

- https://chat.openai.com/share/d0cf78bf-752a-4292-8647-6d0d87c93ab8
- https://chat.openai.com/share/c5e2c17b-c973-4b32-91d1-28b689d2311d
- https://chat.openai.com/share/701a6ff1-a889-4065-aab4-d7e661bf23b4

© Francisco Oliveira Neto - 2018

Consider the example below where the cyclomatic complexity is 5.

```
public String countVowels(String word) {
  int count = 0;
  for (String c : word.split("")) {
    for(String v: vowels) {
      if(c.equalsIgnoreCase(v)) {
        count++;
      }
    }
  }
  if(count == 0) { // +1
    return "does not contain vowels";
  }
  return "contains " + count + " vowels";
}
```

Determine the cyclomatic complexity of the code below.

```
public static void isLeapYear(int year){
  if(year < 0) {
    System.out.println("Year cannot be negative.");

  } else if((year % 4 == 0 && year % 100 != 0) || year % 400 == 0) {
    System.out.println(year + " is a leap year.");

  } else { System.out.println(year + " is a NOT leap year.");}
}
```


© Francisco Oliveira Neto - 2018

Measures: Summary

- There are **many more measures.** Creativity is your limit ☺. Examples:
  - Variations of Cyclomatic complexity. **Def-use pairs** of variables.
  - CK measures for OOP design. Depth of Inheritance Tree (DIT)
  - Fog index: Average length of words and sentences in documents
- Ultimately, measures are associated to Key Performance Indicators (**KPI**) to help us characterise quality attributes.

As QA engineers, focus on attributes and **propose /reuse measures**.

First, define which questions to answer and **then think of KPI.**

© Francisco Oliveira Neto - 2018

Bonus Part:
Code Repositories

© Francisco Oliveira Neto - 2018

Good Software Repositories

- We use **version control systems** (git) to store and track our code.
- Important terms:
  - Git tree
  - Commit
  - Branch
  - Push / Pull:

© Francisco Oliveira Neto - 2018

Good Software Repositories

- We use **version control systems** (git) to store and track our code.
- Important terms:
  - Git tree: The entire history of changes in the code.
  - Commit: A state in the history of changes in the code repository.
  - Branch: An independent series of commits in the repository
  - Push / Pull: Actions to update or retrieve changes in a Git repository.

© Francisco Oliveira Neto - 2018

What is git? How does it work?

- **Version Control System.** Authored by Linus Torvalds in 2005.
- **Stream of snapshots:** Your stream of snapshots is similar to an actual tree (plant).
  - Git stores versions of a work directory as snapshots of current and previous. That is: The current snapshot "includes" the history.
  - You cannot "replace" part of the trunk. While this seems "rigid", this **adds integrity to the data**.

- **`.git`: Nearly Every Operation Is Local.** All the repository is in your local computer.
  - Where can we find more information about the repo? In the local or the remote repo?
  - Git directories are **full-fledged repository**. A **compressed** of the whole history of changes and files.

How to “stand out” as a good git repo?

- For your **Local** repo (also for remote):
  - Use **.gitignore**. Remove hidden trash files such as (.DS_STORE, build or aux files, etc.)
  - Make a **good README** file. This is your welcome card.

- For your **Remote** repo (impress recruiters):
  - Have a **Wiki page** with documentation. Precious for omboarding!
  - Use **traceability**: Milestones, requirements, issues and code reviews.
  - Use **Kanban Boards** **(Todo, Doing, Done)** to show you know about development processes.

https://www.atlassian.com/git/tutorials/comparing-workflows/feature-branch-workflow

Git Workflow

- A git workflow is a set of **rules agreed and followed by the team** on how to use and maintain a git repository. There are numerous workflows available:

- Git Feature Branches Workflow: Changes are branches that cover features.
  1. Many people can work on the same branch. **Name branches** after features.
  2. After completing the feature implementation (i.e., after testing), **you request** that the branch is merged into master.
  3. **Someone reviews** your request and decide whether to accept your changes or ask to implement more.

What is a good commit?

- Use **Atomic Commits**, which is the idea of **small and self-contained code changes**.
- Commit each fix or task as a separate change. These can be in **multiple files**.
- Only commit when a block of work is **complete**. Each commit is a **snapshot!!**
- Atomic Commits are: (i) easier to rollback, and (ii) helps revising merge requests.

- Source: https://www.freshconsulting.com/insights/blog/atomic-commits/
  - by Sean Petterson

What is a good commit message?

- Commit messages communicate **context about a change** to fellow developers.
- A **diff** will tell you **what** changed, **commit** messages tell you **why** it changed.
- **Consistency** is the key ingredient!

- Your team should agree on a commit message convention. Define:
  - **Style:** Markup syntax, wrap margins, grammar, capitalization, punctuation.
  - **Content:** Define the information shared in the message. Also, what should **not** be in the message.
  - **Metadata:** Define whether to include Issue ID, Pull Request number, etc.

Main Rules for a Good Commit Message:

1. Separate subject from body with a blank line
2. Limit the subject line to 50 characters
3. Capitalize the subject line
4. Do not end the subject line with a period
5. Use the imperative mood in the subject line
6. Wrap the body at 72 characters
7. Use the body to explain *what* and *why* vs. *how*

These three are very important rules!

Separate subject from body with a blank line

- Commit messages have a **subject and a body**.
  - `$ git commit –m "Fix typo in introduction to user guide” –m “body”`
- For smaller messages, the subject is enough. But more important changes should include a body. Good examples:
  - This is a good body + header. Even long ones (from Git itself). This one just has a header.
- For adding a body, config your terminal to a text editor. Can be VIM, EMACS, etc.
  - `git config --global core.editor <choose_yours>`

Use the imperative mood in the subject line

- Imperative mood messages **convey a command or instruction**.
- To make it easier, always think of the sentence:

  If applied, this commit will `your subject line here`

- **Examples:**
  - If applied, this commit will `refactor subsystem X for readability`
  - If applied, this commit will `update getting started documentation`
  - If applied, this commit will `merge pull request #123 from user/branch`